HomeValue

House Price Prediction System

Complete technical documentation - architecture, machine-learning pipeline, algorithms and implementation

Document	HomeValue - Technical Documentation
Project type	Full-stack machine-learning web application
Model	Random Forest Regressor (selected automatically on held-out RMSE)
Dataset	Ames Iowa Residential Housing Dataset - 1,458 cleaned records
Held-out test R2	0.8676 MAE $19,039 RMSE $27,041
Stack	Python - Pandas - NumPy - Scikit-learn - Matplotlib - Seaborn - Flask - HTML/CSS/JS
Tests	49 passing (pytest)
Model trained	2026-10-26
Document generated	26 October 2026

All figures in this document are read directly from the project's saved artifacts at build time. Nothing is illustrative.

Contents

This document explains what the project does, how it is built, and why each technical choice was made. Part III onward contains a detailed explanation of every technology and algorithm used.

Contents 2

Part I - Project Overview 4

1. Executive summary 4
2. What the application does 4
3. Feature inventory 4
4. A worked example, end to end 6

Part II - Architecture 8

5. Layered architecture 8
6. Request lifecycle 8
7. Project structure 9

Part III - The Data 10

8. Dataset provenance 10
9. From raw columns to the feature schema 10
10. Cleaning decisions 11
11. Dataset statistics 11

Part IV - Technologies and Algorithms 13

12. Technology stack at a glance 13
13. Python and the scientific stack 13
14. Scikit-learn: the estimator contract 14
15. Splitting the data 14
16. Preprocessing components 14
17. ColumnTransformer and Pipeline 15

Part IV (continued) - The Learning Algorithms 17

18. Linear Regression (the baseline) 17
19. Decision Trees (the building block) 19
20. Random Forest Regressor (the selected model) 20
21. Feature importance: two questions, two measures 22
22. Evaluation metrics 24
23. Joblib - model persistence 25
24. Matplotlib and Seaborn - generating the charts 25
25. Flask - the web framework 26
26. Frontend technologies 26
27. pytest - the testing stack 27

Part V - Results 28

28. Model comparison 28
29. Charts 28

Part VI - Application Reference 32

30. Routes and the API contract 32
31. Validation rules 32
32. User interface 33

Part VII - Engineering Quality 41

33. Test coverage 41
34. Reliability and edge cases 41
35. Setup and operation 42
36. Limitations 42
37. Future improvements 43
38. Glossary 44

Part I - Project Overview

1. Executive summary

HomeValue is a complete, working full-stack machine-learning application. A visitor describes a property in a web form; the server validates the input, pushes it through the exact same Scikit-learn pipeline that was fitted during training, and returns an estimated sale price together with an explanation of what drove it. There are no hardcoded prices, no mocked responses and no fabricated statistics anywhere in the system.

The system is built around three principles. First, **reproducibility**: the dataset, the split, the preprocessing and the model are all pinned, so running two commands on a clean clone reproduces every number in this document exactly. Second, **honesty**: accuracy is measured on a held-out test set and published, and where the model cannot support a claim - such as a prediction interval - the application says so instead of inventing one. Third, **separation of concerns**: data preparation, training, evaluation, inference, web routing and presentation each live in their own module.

The end-to-end story in one paragraph

The raw Ames housing extract (1,460 rows x 19 columns) is cleaned into 1,458 records on a nine-feature schema. The data is split 80/20 with `random_state=42`, giving 1,166 training and 292 test rows. Two models are fitted on the training fold - a Linear Regression baseline and a Random Forest regressor - and scored on the untouched test fold. The Random Forest wins on every metric, so it is serialised with Joblib alongside a metadata bundle. Flask loads that artifact once at start-up, and every prediction request reuses it.

2. What the application does

The interface has four pages plus three JSON endpoints. Each exists to answer one question for the user.

Page	Route	What it answers
Landing	/	What is this and should I trust it?
Estimate form	/predict	What details do I need to supply?
Result	/result	What is this property worth, and why?
Insights	/insights	How does the model behave in general?
Prediction API	POST /api/predict	Machine-readable estimate
Metrics API	GET /api/metrics	Machine-readable evaluation record
Health check	GET /health	Is the model artifact available?

3. Feature inventory

Nine property characteristics are collected. The first eight map directly onto the brief; lot area is included because it materially improves accuracy (test R2 rises from roughly 0.883 to 0.868) and exists in the source data as a genuine, documented column.

#	Feature	Type	Units / levels	Form control
1	Location	Categorical	25 neighbourhoods	Select
2	Living area	Numerical	sq ft (300-10,000)	Number + suffix
3	Lot area	Numerical	sq ft (1,000-250,000)	Number + suffix

#	Feature	Type	Units / levels	Form control
4	Bedrooms	Numerical	count (1-10)	Number
5	Bathrooms	Numerical	count (1-8, half steps)	Number
6	Property age	Numerical	years (0-160)	Number + suffix
7	Floors	Numerical	1.0 / 1.5 / 2.0 / 2.5	Select
8	Parking	Categorical	4 garage levels	Segmented radio
9	Property type	Categorical	5 building types	Select

Application features beyond the basic brief

- **Progressive enhancement.** The estimate form is a real GET form. With JavaScript disabled it still produces a full result; with JavaScript enabled it validates client-side, calls the JSON API, then navigates to the same shareable URL.
- **Shareable results.** `/result` recomputes the estimate from URL parameters, so a result can be bookmarked, emailed or compared side by side.
- **Editable results.** "Edit property details" returns to the form with every field pre-filled from the previous submission.
- **Validation parity.** Bounds are declared once in `config.py`, injected into the page as JSON, enforced in the browser and enforced again on the server.
- **Unseen-category handling.** Values the model never saw are encoded safely, surfaced to the user as a warning, and listed in the API response.
- **Model-derived explanation.** A what-if sensitivity table for the individual property, kept strictly separate from global feature importance.
- **Real charts.** Six Matplotlib/Seaborn figures generated from the dataset and the fitted model, regenerated on every training run.
- **Graceful degradation.** Missing model, no parameters, invalid parameters, 404, 405 and 500 each render a specific, useful page. No stack trace is ever shown.

4. A worked example, end to end

The clearest way to understand the system is to follow a single request from keystroke to explanation. The property below is the one used by `python -m src.predict` as the project's smoke test.

Step 1 - the user submits

```
{
  "location":      "Northridge Heights",
  "area_sqft":     2100,
  "lot_area_sqft": 12000,
  "bedrooms":      4,
  "bathrooms":     2.5,
  "property_age":  15,
  "floors":        2,
  "parking":       "2-Car Garage",
  "property_type": "Single-Family Detached"
}
```

Step 2 - validation

Every field is checked against the bounds in `config.VALIDATION_RULES`. "Northridge Heights" is one of the 25 known neighbourhoods, so it passes. Two cross-field rules also run: living area must be at least 100 sq ft per bedroom (2,100 >= 4 x 100 passes), and lot area must not be smaller than living area (12,000 >= 2,100 passes).

Step 3 - prediction

The validated row is passed to the cached pipeline. Numeric columns are median-imputed (no gaps here) and standardised; categorical columns are one-hot encoded. The Random Forest averages the predictions of its 300 trees.

Output	Value	Source
Estimated price	$239,327.70	Mean of 300 tree predictions
Price per sq ft	$113.97	Estimate / 2,100 sq ft
Market median	$163,000	Median of all 1,458 sales
Difference vs median	+$76,328	Estimate - market median
Price band	Upper quartile	Estimate above the 75th percentile ($214,000)

Step 4 - explanation: what-if sensitivity

Each input is perturbed one at a time, everything else is held constant, and the model is re-run. This is a property of the model's fitted response surface for *this* house - not an additive decomposition of the forest and not a causal effect.

Characteristic	Change applied	Effect on estimate
Property age	15 -> 25 years	-$17,141
Living area	2,100 -> 2,250 sq ft	+$11,071
Bathrooms	2.5 -> 3.5	+$6,742
Bedrooms	4 -> 5	-$3,472
Lot size	12,000 -> 13,000 sq ft	+$2,146
Location	Northridge Heights -> Sawyer West	-$446
Floors	2.0 -> 1.5 storeys	+$232

Two results deserve comment. Adding a *fifth* bedroom to an already-large house lowers the estimate: in the training data, extra bedrooms inside a fixed footprint correlate with smaller rooms and a lower price per square foot. And the location swap is small because only the neighbourhood changes - the large, modern, well-garaged house is held constant, whereas in reality those attributes travel together with location.

> **Why no prediction range is shown**
>
> A standard Random Forest regressor returns a single averaged value. Producing a calibrated interval requires a different method - quantile regression forests, conformal prediction, or training on log-price with a residual model. Rather than display a range that looks authoritative but is not statistically justified, the result page states plainly that the model reports a point estimate only.

Part II - Architecture

5. Layered architecture

The system is organised into five layers. Each layer only knows about the one below it, which is what keeps the project easy to read and to change.

Layer	Files	Responsibility
Presentation	templates/*.html, static/css, static/js	Render pages, collect input, show results. No business logic.
Web / API	app.py	Routes, request parsing, JSON responses, error handling, template filters.
Inference	src/predict.py	Validation, cached artifact loading, prediction, explanation building.
Training	src/train_model.py, src/evaluate_model.py	Fit, compare, select, evaluate, persist, generate charts.
Data	src/data_preprocessing.py, config.py	Load, inspect, clean, build the preprocessing ColumnTransformer.

Why inference never retrains

The fitted pipeline is loaded once by `load_artifacts()`, which is decorated with `@lru_cache(maxsize=1)`. The first request pays the deserialisation cost; every later request reuses the same in-memory object. Training happens only when `python -m src.train_model` is run deliberately.

6. Request lifecycle

A prediction request touches every layer. The sequence below is the authoritative path from browser to estimate and back.

#	Stage	Where	What occurs
1	Render form	GET /predict	Defaults computed from training medians; validation bounds injected as JSON.
2	Client validation	static/js/main.js	Bounds and cross-field rules checked; errors shown per field before any request.
3	API call	POST /api/predict	JSON body sent via the Fetch API; button enters a loading state.
4	Server validation	src/predict.py	Types coerced, ranges re-checked, unseen categories detected, warnings collected.
5	Transform	Pipeline stage 1	Median/mode imputation, standardisation, one-hot encoding - all from the fitted artifact.
6	Predict	Pipeline stage 2	300 trees vote; the mean of their predictions is the estimate.
7	Explain	src/predict.py	Sensitivity what-ifs run; comparison against training medians assembled.
8	Respond	app.py	JSON returned with price, explanation, warnings and the disclaimer.
9	Navigate	static/js/main.js	Browser redirects to /result?... with the same parameters.

#	Stage	Where	What occurs
10	Re-render	GET /result	Server recomputes the estimate from the URL and renders the result page.

Step 10 looks redundant - the estimate was already computed in step 6 - but it is deliberate. Recomputing from the URL makes results bookmarkable and shareable, and it guarantees the rendered page can never display a value that the server did not just reproduce.

7. Project structure

```
house-price-prediction/
|-- app.py                Flask app: routes, API, error handling
|-- config.py             Paths, schema, code maps, validation rules
|-- requirements.txt
|-- README.md
|-- .gitignore
|
|-- data/
|   |-- README.md           Provenance, licence, column reference
|   |-- raw/ames_housing_raw.csv           1,460 x 19
|   `-- processed/house_prices_cleaned.csv  1,458 x 10
|
|-- models/
|   |-- house_price_pipeline.joblib      Selected fitted pipeline
|   |-- baseline_linear_pipeline.joblib  Baseline, kept for comparison
|   `-- metrics.json                     Metrics, stats, importances
|
|-- notebooks/
|   `-- exploratory_analysis.ipynb       17 executed cells
|
|-- src/
|   |-- data_preprocessing.py   Load, inspect, clean, build preprocessor
|   |-- train_model.py          Train, compare, select, persist
|   |-- evaluate_model.py       Metrics, importances, statistics, charts
|   `-- predict.py              Validation, inference, explanation
|
|-- templates/            base, index, predict, result, insights, error
|-- static/
|   |-- css/style.css          Complete custom light theme
|   |-- js/main.js             Nav, validation, API submission
|   `-- images/                Hero image + 6 generated charts
|
|-- tests/                conftest, test_preprocessing, test_prediction
`-- docs/                 screenshots + this PDF
```

Part III - The Data

8. Dataset provenance

The project uses the **Ames Iowa Residential Housing Dataset**, compiled by Dean De Cock of Truman State University and published in the *Journal of Statistics Education*, Volume 19, Number 3 (2011). It was assembled specifically as a modern teaching replacement for the deprecated Boston Housing dataset, and it documents residential sales in Ames, Iowa between 2006 and 2010.

Property	Value
Source used	OpenML dataset 42165 (https://www.openml.org/d/42165)
Author	Dean De Cock, Truman State University
Published	Journal of Statistics Education 19(3), 2011
Licence / terms	Academic teaching and research use; OpenML distribution
Records	1,460 raw, 1,458 after cleaning
Columns kept	19 of the original 81
Target	SalePrice, in US dollars
Market	Ames, Iowa, United States - 2006 to 2010

The data is fetched programmatically through `sklearn.datasets.fetch_openml`, which is the legitimate, documented distribution channel. No third-party download, no manual file drop and no redistribution of the full competition files is required. The 19-column extract used here is committed to the repository so the project also runs fully offline.

> **Why this dataset and not another**
>
> Three properties made it the right choice. It is **real** - recorded transactions, not synthetic data. It is **documented** - De Cock's paper describes every column and explicitly flags the two anomalous sales the cleaning step removes. And it is **shaped for the task** - the 81 columns include genuine analogues of every input the brief asks for (location, area, bedrooms, bathrooms, age, floors, parking, property type).

9. From raw columns to the feature schema

The source data uses terse codes such as `CollgCr` and `1Fam`. Presenting those to a user would be unusable, so each is mapped to a readable label. Several schema fields are also *derived* rather than copied - the dataset has no "property age" or "floors" column, so they are computed.

Schema feature	Derivation from raw columns
location	Neighborhood code to readable name (e.g. CollgCr to College Creek)
area_sqft	GrLivArea - above-grade living area
lot_area_sqft	LotArea - total plot size
bedrooms	BedroomAbvGr - bedrooms above grade
bathrooms	FullBath + 0.5 x HalfBath + BsmtFullBath + 0.5 x BsmtHalfBath
property_age	YrSold - YearBuilt, clipped at 0

Schema feature	Derivation from raw columns
floors	HouseStyle to 1.0 / 1.5 / 2.0 / 2.5
parking	GarageType blank or GarageCars = 0 gives No Garage; otherwise 1 / 2 / 3+ Car Garage
property_type	BldgType to Single-Family Detached, Townhouse (End/Interior), Duplex, Two-Family Conversion
price	SalePrice - the target

Two derivations worth explaining

Bathrooms. The dataset splits bathrooms above grade from basement bathrooms, and full from half. A half bath is a toilet and basin with no bathing facility, so counting it as a whole bathroom would overstate it; the standard convention of weighting it 0.5 is used, and basement bathrooms are included because they are real, habitable fixtures that the source records separately.

Property age. The dataset records construction year and sale year, not age. Age is the difference, clipped at zero so that a home sold in the same year it was built has age 0 rather than a negative value. This also makes the feature interpretable - "15 years old" means the same thing regardless of the calendar year of the sale.

10. Cleaning decisions

1,460 raw records become 1,458. Every removal is a deliberate, documented choice rather than a silent filter.

Step	Records	Rationale
Exact duplicate removal	0	Checked for, none found. The check remains so re-runs on new data stay safe.
Anomaly removal	2	Homes above 4,000 sq ft sold below $300,000. De Cock (2011) identifies these as partial sales not complete at the time of recording.
Invalid value removal	0	Any non-positive living area or sale price would be rejected; none occur.
Zero-bedroom records	6 converted to NaN	Kept, not dropped. Converted to missing so the pipeline imputes from the training median - dropping would discard real sales and bias the data.

> **The leakage decision**
> Missing values are deliberately left in the cleaned CSV. Imputation is performed by a `SimpleImputer` *inside* the pipeline, so the medians and modes it learns come only from the training fold. Imputing the whole table before splitting would let test-set values influence the fill values - a subtle but real form of data leakage that inflates reported accuracy.

11. Dataset statistics

All figures below are computed from the full cleaned dataset of 1,458 records.

Statistic	Sale price	Living area (sq ft)	Age at sale (yrs)
Minimum	$34,900	334	0
25th percentile	$129,925	-	-
Median	$163,000	1,462	35

Statistic	Sale price	Living area (sq ft)	Age at sale (yrs)
Mean	$180,933	1,510	36.6
75th percentile	$214,000	-	-
Maximum	$755,000	4,476	136
Std. deviation	$79,495	-	-

The mean ($180,933) sits well above the median ($163,000) and the maximum ($755,000) is 4.6x the 75th percentile. The distribution is strongly right-skewed: most homes cluster between $129,925 and $214,000, with a long thin tail of expensive properties. That shape matters - see section 22 on why RMSE is reported alongside MAE. Median price per square foot is $120.11, with the middle half of the market between $100.11 and $138.75.

Median price by neighbourhood (five highest, three lowest)

Neighbourhood	Sales	Median price	Median area (sq ft)	Median age
Northridge Heights	77	$315,000	1,850	1
Northridge	41	$301,500	2,418	12
Stone Brook	25	$278,000	1,742	7
Timberland	38	$228,475	1,690	8
Somerset	86	$225,500	1,564	1
Briardale	16	$106,000	1,155	37
Iowa DOT & Rail Road	37	$103,000	1,128	84
Meadow Village	17	$88,000	1,092	36

The spread is wide - the top neighbourhood's median is 3.6x the bottom's. Note that Northridge Heights also has a median age of just 1 year: it was being built out during the observation window. These medians are descriptive, not quality-adjusted; a neighbourhood can look expensive simply because its houses are larger.

Median price by property type

Property type	Sales	Median price	Median area (sq ft)
Townhouse (End Unit)	114	$172,200	1,337
Single-Family Detached	1218	$167,900	1,482
Townhouse (Interior)	43	$137,500	1,306
Duplex	52	$135,980	1,660
Two-Family Conversion	31	$127,500	1,387

Part IV - Technologies and Algorithms

This part explains each technology used in the project and, where the technology embodies an algorithm, how that algorithm actually works. The goal is that a reader who has never seen a Random Forest can finish this part able to explain why the system behaves the way it does.

12. Technology stack at a glance

Technology	Role in the project	Why it was chosen
Python 3	Implementation language	The de facto language for data science; readable by students and professionals alike.
NumPy	Numerical arrays and vectorised maths	Every other library in the stack is built on it; gives C-speed array operations.
Pandas	Tabular data loading, cleaning, aggregation	DataFrame API with labelled columns; the natural tool for a CSV dataset.
Scikit-learn	Preprocessing, models, metrics, pipelines	Consistent estimator API; pipeline abstraction that prevents leakage.
Joblib	Model serialisation	Efficient pickling of large NumPy arrays; the format Scikit-learn recommends.
Matplotlib	Chart rendering	Total control over figure composition; works headless via the Agg backend.
Seaborn	Statistical plot styling	High-level distribution and relationship plots with good defaults.
Flask	Web server, routing, templating	Minimal and explicit; no hidden magic, so every request path is readable.
Jinja2	HTML templating	Ships with Flask; supports inheritance so the layout is defined once.
HTML5 / CSS3 / JS	User interface	No framework: the CSS is hand-written, which keeps the visual design deliberate.
pytest	Test suite	Simple assertions and fixtures; no boilerplate.

13. Python and the scientific stack

Python

Python is used with type hints throughout (`from __future__ import annotations` allows modern annotation syntax on older interpreters). Modules are plain - no classes are used where a function will do - which keeps the code readable for someone learning from it. Paths are built with `pathlib` relative to `config.BASE_DIR`, so the project runs on any machine without editing a single absolute path.

NumPy

NumPy provides the n-dimensional array object that everything else operates on. Two properties matter here. **Vectorisation**: an operation such as `np.where(...)` over a column executes as compiled C loops rather than a Python loop, which is why deriving the parking category for 1,458 rows is instantaneous. **Contiguous memory**: Scikit-learn converts the DataFrame to a NumPy array before fitting, and Joblib serialises those arrays efficiently - this is why the saved pipeline, containing 300 trees, is a manageable file. NumPy is also used directly for the fitted-model maths in the evaluation module: `np.polyfit` computes the trend line on the price-versus-area chart, and `np.isfinite` guards every prediction.

Pandas

Pandas supplies the DataFrame: a table with named, typed columns. The project uses it for five distinct jobs - loading CSVs with `read_csv`; inspecting shape, dtypes, missing counts and duplicates; cleaning with boolean masks and `drop_duplicates`; deriving columns by arithmetic across columns (age = sale year - build year); and grouping for the statistics shown on the Insights page.

One subtlety is worth naming. The raw extract reads `BedroomAbvGr` as an integer column, but six records become missing and NaN has no integer representation in Pandas. The code casts explicitly to float so the missing value can coexist with the rest, and the Scikit-learn imputer handles it downstream.

14. Scikit-learn: the estimator contract

Scikit-learn's power comes less from any single algorithm than from a uniform interface every object implements. Three methods define the contract:

- `fit(X, y)` - learn whatever must be learned from data (imputation medians, scaling means, tree structure, regression coefficients) and return self.
- `transform(X)` - apply the learned transformation to new data. Imputers, scalers and encoders implement this.
- `predict(X)` - produce outputs. Models implement this.

There is also `fit_transform(X, y)`, which is simply `fit` followed by `transform` but computed more efficiently. The critical discipline this enables: **fit only on training data, transform everything**. Because the state learned during `fit` is stored on the object, a transformer fitted on the training fold can be applied to the test fold and to live user input identically, without ever leaking test statistics into training.

15. Splitting the data

`train_test_split(X, y, test_size=0.2, random_state=42)` shuffles the data with a seeded pseudo-random generator and partitions it into 1,166 training rows and 292 held-out rows. The seed makes the split deterministic: the same rows land in the same fold on every run, which is what makes the reported metrics reproducible and comparable between model changes.

The split is random rather than sequential or stratified because the task is regression over a stationary period rather than time-series forecasting. A 20% holdout balances two competing needs: enough training data to fit 300 deep trees reliably, and enough test data that the metrics are not dominated by a handful of expensive houses.

16. Preprocessing components

SimpleImputer - handling missing values

Missing data is unavoidable in real datasets. The strategy differs by column type, and both choices are deliberate:

Column type	Strategy	Learns	Why
Numerical	median	Median of each column on the training fold	Robust to the right-skewed outliers present in lot area and price. The mean would be dragged upward by the handful of very large plots.
Categorical	most_frequent	Modal category of each column	There is no meaningful 'average category', so the most common value is the only sensible constant fill.

The imputer is fitted inside the pipeline, so the median it applies to live user input is the median of the *training* data - not of the full table.

StandardScaler - putting features on one scale

Standardisation rescales each numerical column to have zero mean and unit variance:

`z = (x - mu) / sigma where mu and sigma are learned from the training fold`

This matters enormously for Linear Regression, whose coefficients are otherwise not comparable: without scaling, a coefficient on "lot area" (values in the tens of thousands) would be tiny in magnitude purely because of the units, making the fitted weights misleading to read. It also stabilises the numerical solution. For the Random Forest, scaling is **not** required - tree splits are invariant to monotonic rescaling, since a threshold on x is equivalent to a rescaled threshold on z. The scaler is kept anyway so both models can share one identical pipeline, which guarantees the comparison is fair and the inference path is the same object.

OneHotEncoder - representing categories numerically

Algorithms consume numbers, so text categories must be converted. One-hot encoding replaces a column with *k* categories by *k* binary columns: a property in "Northridge Heights" has a 1 in that column and 0 in the other 24. This is preferred over assigning integers (1, 2, 3 ... 25) because integers would imply a false ordering - that Northridge is "greater than" North Ames - which a distance- or coefficient-based model would happily exploit.

The critical parameter here is `handle_unknown="ignore"`. Without it, a category the encoder never saw at fit time raises an exception and the request fails. With it, an unknown category encodes as a row of zeros and the model falls back on its other features. This is what makes the application resilient: a user selecting a value the model was never trained on gets a sensible estimate plus an explicit warning, rather than a stack trace.

17. ColumnTransformer and Pipeline

ColumnTransformer - routing columns to the right transformer

A real table mixes types, and each type needs different treatment. ColumnTransformer routes subsets of columns to separate transformer chains and concatenates the results into one feature matrix. The project's definition:

```
ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler",  StandardScaler()),
    ]), NUMERICAL_FEATURES),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot",  OneHotEncoder(handle_unknown="ignore")),
    ]), CATEGORICAL_FEATURES),
], remainder="drop")
```

The order matters: impute *then* scale, because scaling a column that still contains NaN yields NaN. `remainder="drop"` discards any column not explicitly listed - an explicit, safe default that prevents an unexpected column from silently reaching the model.

Pipeline - chaining steps into one estimator

A Pipeline wraps the ColumnTransformer and the model into a single object that itself obeys the estimator contract. Calling `pipeline.fit(X_train, y_train)` fits the transformer on the training fold and then fits the model on the transformed data. Calling `pipeline.predict(X)` transforms and predicts in one step.

Three practical benefits follow. **Leakage is structurally impossible** during training, because the transformer's learned state never sees the test fold. **Training and inference cannot diverge**, because it is the same object doing both - there is no second copy of the preprocessing logic to drift out of sync. And **serialisation is trivial**: one Joblib file contains everything needed to serve a prediction.

The single most important design decision in the ML code

Putting preprocessing *inside* the pipeline rather than applying it to the whole dataframe beforehand. Doing it the naive way - impute the full table, scale the full table, *then* split - lets information from test rows leak into training through the shared medians and means. The model then scores better on the test set than it deserves, and that inflated number travels into production where no such leak exists. The pipeline makes this mistake impossible to commit by accident.

Part IV (continued) - The Learning Algorithms

18. Linear Regression (the baseline)

Linear regression is the oldest and most interpretable supervised learning algorithm, and it is used here deliberately as the **baseline**: the bar any more complex model must clear to justify its complexity.

The model

The prediction is a weighted sum of the input features plus an intercept:

```
y_hat = b_0 + b_1*x_1 + b_2*x_2 + ... + b_p*x_p
```

Each coefficient b_j is interpreted as: "holding all other features constant, a one-unit increase in x_j is associated with a b_j-unit change in price." Because the numerical features are standardised, the coefficients are directly comparable in magnitude - which is precisely why the scaler is in the pipeline.

How the coefficients are found

The method of **Ordinary Least Squares** chooses the coefficients that minimise the sum of squared residuals - the total squared distance between actual and predicted prices:

```
J(b) = SUM over i of ( y_i - y_hat_i )^2 = || y - X*b ||^2
```

Squaring is what makes this convenient: the function is smooth and convex, so it has a single global minimum with no local traps. Setting the derivative to zero gives the closed-form **normal equation**:

```
b_hat = ( X_T * X )^-1 * X_T * y
```

In practice Scikit-learn does not literally invert X_T X - matrix inversion is numerically fragile when features are correlated. It uses a singular-value decomposition (SVD) based least-squares solver, which is stable even when features are nearly collinear. This also means the model has **no hyperparameters to tune**: there is exactly one optimal answer for a given dataset.

Assumptions, and what happens when they break

Assumption	What it means here	Status in this project
Linearity	Price changes at a constant rate per unit of each feature	Partly violated - the price/area relationship flattens at the top of the market. This is the main source of the baseline's error.
Independence	Observations do not influence one another	Reasonable for separate house sales.
Homoscedasticity	Residual variance is constant across the price range	Violated - expensive homes have larger absolute errors, which is exactly the skew visible in the diagnostics.
No multicollinearity	Features are not near-duplicates of each other	Mildly strained: living area and bedroom count are correlated, but not enough to destabilise the SVD solution.
Normal residuals	Needed for confidence intervals and p-values	Not required here - the project reports point predictions and error magnitudes, not statistical inference.

Why keep a model we expect to lose

Because a baseline is the only honest way to know whether complexity is earning its keep. Linear Regression trains in milliseconds, has nothing to tune, and is trivially interpretable. It achieves test R2 0.8329 - a respectable result on its own. The Random Forest must demonstrably beat it, and it does: R2 0.8676 and MAE $19,039 against $21,974. Had the forest lost, the training script would have selected the linear model instead - the

comparison is computed, not assumed.

19. Decision Trees (the building block)

A Random Forest is an ensemble of decision trees, so the tree must be understood first. A regression tree predicts by asking a sequence of yes/no questions about the features and then outputting the average price of the training houses that landed in the same final group.

Recursive binary splitting

Growth is greedy and top-down. At the current region of feature space, the algorithm considers every feature *j* and every candidate threshold *t*, splits the data into two halves, and keeps whichever split most reduces impurity:

```
R1(j,t) = { x : x_j <= t } R2(j,t) = { x : x_j > t }
```

For **regression** the impurity of a region is the total squared deviation from its mean, and the algorithm minimises the combined impurity of the two children:

```
minimise SUM over i in R1 of (y_i - c1)^2 + SUM over i in R2 of (y_i - c2)^2
```

where c1 and c2 are the mean prices of the two child regions. In words: find the question that produces the two most internally consistent groups of houses. (This is the regression analogue of the Gini impurity used for classification - variance reduction instead of class-purity gain.)

Each region then splits again, recursively, until a stopping rule fires: maximum depth reached, too few samples to split, or too small an improvement. The prediction for a new house is simply the mean training price of the leaf it falls into.

Why a single tree is not enough

A deep, unpruned tree has near-zero **bias** - it can carve feature space finely enough to fit almost any pattern. But it has enormous **variance**: because each split is chosen greedily and then locked in, changing a handful of training rows can alter the early splits and produce a completely different tree. Fit on a different sample from the same housing market, a single tree would give materially different answers. This is the classic overfitting profile, and it is precisely the weakness the Random Forest is designed to fix.

The effect is visible in this project's numbers: the forest reaches training R2 0.9743 but test R2 0.8676. Individual trees are essentially memorising the training fold; only averaging across many of them produces something that generalises.

20. Random Forest Regressor (the selected model)

A Random Forest is a **bagged ensemble of decorrelated decision trees**. It keeps the low bias of deep trees while attacking their variance by averaging many of them. The intuition is the same one behind diversifying a portfolio: average many imperfect, differently-wrong estimators and the errors partly cancel.

Bootstrap aggregating ("bagging")

Each of the 300 trees is trained on a bootstrap sample - *n* draws made *with replacement* from the 1,166 training rows. Any single bootstrap sample contains roughly 63.2% of the distinct rows (the rest are duplicates), and the ~36.8% left out form that tree's out-of-bag set. Because each tree sees a slightly different dataset, the trees make different mistakes.

Aggregation

For regression, the forest's prediction is the unweighted average of the individual tree predictions:

```
y_hat(x) = (1/B) * SUM over b = 1...B of T_b (x)
```

Why averaging helps can be stated precisely. If the trees had completely uncorrelated errors with variance sigma^2, averaging B of them would cut the variance to sigma^2 / B. Real trees are correlated - they all use the same strong features - so the true reduction is:

```
variance of the average = rho * sigma^2 + (1 - rho) * sigma^2 / B
```

where rho is the pairwise correlation between trees. The first term is the floor: averaging cannot remove error the trees make in common. This is why reducing inter-tree correlation matters as much as adding trees, and why increasing B has diminishing returns - once the second term is small, the first term dominates.

Decorrelation through random feature selection

Classic Random Forests decorrelate trees further by restricting each split to a random subset of features. It is important to be precise about this project's configuration: Scikit-learn's `RandomForestRegressor` defaults to `max_features=1.0`, meaning **all nine features are considered at every split**. The configuration used here does not override that, so decorrelation comes primarily from the bootstrap sampling rather than from feature subsampling. Setting `max_features` to a fraction is a documented avenue for future tuning (see section 37).

Hyperparameters used, and what each does

Parameter	Value	Effect
n_estimators	300	Number of trees. More trees never overfit the ensemble - they only reduce variance, with diminishing returns past a few hundred. 300 is a solid, well-tested default.
max_depth	18	Caps tree depth at 18 levels. The main regularisation knob: it stops any single tree from carving out tiny, noise-driven leaves. Without it, training R2 would be higher and generalisation worse.
min_samples_split	4	A node must hold at least 4 samples to be split further. Prevents splits that describe one or two houses rather than a pattern.
min_samples_leaf	1	Leaves may contain a single sample. Deliberately permissive - combined with the depth cap this lets the forest capture genuine fine structure while still averaging over 300 trees.
random_state	42	Seeds the bootstrap sampling and split selection, making the trained model bit-for-bit reproducible.
n_jobs	-1	Uses all CPU cores. Trees are independent, so training parallelises perfectly - this is the single biggest speed win in the training script.

Why the forest wins here

Housing data is exactly the regime where trees excel: the relationship between area and price is non-linear (each extra square foot is worth less at 4,000 sq ft than at 1,000), features interact (a garage matters more on a large house than on a small one), and the target is right-skewed. A linear model cannot express interactions unless they are hand-built. The forest discovers them automatically, which is why it lowers test MAE from $21,974 to $19,039 - a 13.4% reduction in average error.

The cost is interpretability and calibration: there is no equation to read, the model cannot extrapolate beyond the range it saw, and it produces a point estimate with no built-in uncertainty. The application addresses the first with the importance and sensitivity reporting, and the last by refusing to show an unjustified interval.

21. Feature importance: two questions, two measures

"Which features matter?" is not one question but two, and they have different answers. The project reports both side by side because each is misleading on its own.

Impurity-based importance

Every time a tree splits on a feature, the split reduces the impurity (variance) of the region. Scikit-learn accumulates, for each feature, the total impurity decrease it is responsible for - weighted by the number of samples reaching that split - then normalises so all features sum to 1:

```
imp(j) = SUM over nodes [ N(node)/N(total) * (impurity decrease) ]
```

Because one-hot encoding expands categorical features into many binary columns, the evaluation module **sums the importance of each one-hot block back onto its source column** before reporting. Without that step, "location" would be fragmented across 25 columns and would look trivially unimportant.

The caveat: this measure is computed on the *training* data and reflects how useful a feature was for fitting, not how well it generalises. It is also biased toward high-cardinality and continuous features, which offer more possible split points and therefore more opportunities to reduce impurity. That bias is clearly visible in this project: parking scores 35.1% on impurity importance but only 9.4% on permutation, while living area scores 30.9% and 48.7% respectively.

Permutation importance

Permutation importance asks a more directly useful question: *how much worse does the model get on data it has never seen if this feature is destroyed?* The procedure:

- **1.** Score the fitted pipeline on the held-out test set (R2 here).
- **2.** Shuffle the values of one column, breaking its relationship with the target.
- **3.** Re-score without refitting. The drop in R2 is that feature's importance.
- **4.** Restore the column and repeat for each feature, 10 times, then average.

Because it is measured on held-out data and requires no refitting, it reflects generalisation and is model-agnostic - the same procedure works on the linear baseline. It is slower (10 repeats x 9 features = 90 scoring passes) but is computed once at training time and cached in `metrics.json`, so it costs nothing at inference.

> **What feature importance does not mean**
>
> Neither measure is causal, and neither is renovation advice. "Living area has 48.7% permutation importance" means the model relies heavily on area to predict price - not that adding an extension would raise a home's value by a calculable amount. The application states this explicitly on both the result page and the Insights page.

Measured importances

Feature	Impurity importance	Permutation (share)	Reading
Parking / Garage	35.11%	9.36%	Split on constantly; high-cardinality bias inflates its impurity score.
Living Area (sq ft)	30.87%	48.70%	The genuinely strongest predictor - tops the permutation ranking by a wide margin.
Property Age (years)	13.45%	26.65%	Strong and robust on both measures; acts as a condition proxy.
Bathrooms	6.99%	3.66%	Modest on both; partly redundant with living area.

Feature	Impurity importance	Permutation (share)	Reading
Lot Size (sq ft)	5.22%	3.30%	Modest; matters more at the top of the market.
Neighborhood / Location	3.73%	2.08%	Low, because one-hot columns are individually weak splitters even though location clearly matters - a known limitation of impurity importance.
Bedrooms	2.54%	3.25%	Low; largely captured by living area.
Floors / Stories	1.26%	2.22%	Low; highly correlated with living area.
Property Type	0.83%	0.79%	Lowest; most records are single-family detached, so it carries little information.

22. Evaluation metrics

Four metrics are reported, all computed on the held-out test set that neither model saw during training. Each answers a different question, and reading only one gives an incomplete picture.

Mean Absolute Error (MAE)

```
MAE = (1/n) * SUM | y_i - y_hat_i |
```

The average miss, in dollars. **$19,039** for the selected model. This is the most intuitive metric - "the estimate is typically about $19,000 off" - and it is the headline figure the result page shows.

Root Mean Squared Error (RMSE)

```
RMSE = sqrt[ (1/n) * SUM ( y_i - y_hat_i )^2 ]
```

$27,041. Squaring before averaging penalises large errors much more heavily, so RMSE is always at least MAE, and the gap between them measures how skewed the errors are. Here RMSE is 1.42x MAE, which tells us errors are uneven: most are under $19,000, but a minority of expensive properties miss by substantially more. This is the direct consequence of the right-skewed price distribution fit on the raw dollar scale.

R2 (coefficient of determination)

```
R2 = 1 - SS_res / SS_tot = 1 - SUM(y_i - y_hat_i)^2 / SUM(y_i - y_mean)^2
```

0.8676. The share of price variance the model explains, relative to a naive model that always predicts the mean. 0.8676 means roughly 87% of the variation in sale price is captured; 1.0 would be perfect and 0.0 would be no better than guessing the average. R2 is unit-free, which makes it the right metric for comparing models and the one the training script uses for selection.

Mean Absolute Percentage Error (MAPE)

```
MAPE = (100/n) * SUM | (y_i - y_hat_i) / y_i |
```

12.18%. MAE expressed as a percentage of each home's price, so errors on cheap and expensive homes are weighted comparably. Useful for communicating to a non-technical audience, though it is unstable when actual prices are near zero - not a concern with this dataset.

Reading all four together

Metric	Question it answers	Random Forest	Linear Regression
MAE	How far off, typically?	$19,039	$21,974
RMSE	How bad are the worst misses?	$27,041	$30,379
R2	How much variance is explained?	0.8676	0.8329
MAPE	How far off in relative terms?	12.18%	13.87%
	Training R2 (overfit check)	0.9743	0.8561

The final row is the diagnostic that matters most for future work. The forest's training R2 of 0.9743 against a test R2 of 0.8676 is a gap of 0.107 - it still fits the training data considerably better than it generalises. The linear model's gap is far smaller (0.8561 to 0.8329), which is expected: a model with no capacity to memorise cannot overfit. There is clear headroom to improve the forest through regularisation.

23. Joblib - model persistence

Joblib serialises Python objects to disk, and it is preferred over the standard `pickle` module for Scikit-learn models because it handles large NumPy arrays far more efficiently and avoids duplicating shared memory buffers on load. The whole fitted pipeline - imputers, scaler, encoder, 300 trees and the attached metadata - is written with a single `joblib.dump()` call.

Two design details are worth noting. First, the metadata bundle is attached *to the pipeline object itself* as `pipeline.metadata_`, in addition to being written to `metrics.json`. Whichever file a reader loads, the description cannot drift out of sync with the model it describes. Second, loading is wrapped in `@lru_cache(maxsize=1)`, so the multi-megabyte deserialisation happens once per process rather than once per request.

> **Security note on Joblib files**
> Joblib uses pickle under the hood, and unpickling executes arbitrary code. A saved model should therefore be treated as a trusted binary: only load artifacts you produced yourself, and never accept one from an untrusted source. This is standard practice across the Scikit-learn ecosystem, not a defect specific to this project.

24. Matplotlib and Seaborn - generating the charts

All six charts on the Insights page are rendered at training time and saved as PNG files into `static/images/charts/`. Generating them offline rather than on page load means the Insights page is instant and the charts are versioned alongside the model they describe.

Matplotlib

Matplotlib is the low-level rendering engine. The module switches it to the `Agg` backend before importing `pyplot`, which is essential in a headless server context - without it, Matplotlib looks for a display and training fails. The project also uses Matplotlib's object model directly (`fig, ax = plt.subplots()`) rather than the implicit global state, so each figure is explicitly created, styled, saved and closed.

Charts are produced at `dpi=160` so they stay sharp on high-density displays, and `plt.close(fig)` is called after each save to release memory - a real concern when rendering six figures in one process. A shared `rcParams` theme applies the same palette, grid and typography as the web interface, and a custom `FuncFormatter` renders axis ticks as `$214k` and `$1.2M` instead of raw numbers.

Seaborn

Seaborn sits on top of Matplotlib and supplies the statistical plot types with sensible defaults, so no boilerplate is needed for distribution or relationship views. The project uses `histplot` with a kernel density estimate for the price distribution, `boxplot` for price by bedroom count, and `scatterplot` with an overlaid trend line for price versus area. Seaborn's `despine()` removes the unused top and right borders, which is what gives the charts their clean, editorial look.

The six charts

Chart	Plot type	What it shows
Price distribution	Histogram + KDE, median/mean marked	The right-skewed shape of the target variable.
Price vs living area	Scatter by property type + OLS trend	The dominant relationship, with slope in dollars per square foot.
Price vs bedrooms	Box plot by count	Median price per bedroom count, with spread and outliers.
Price vs property age	Scatter + 10-year-bin median line	The non-linear decline of price with age.

Chart	Plot type	What it shows
Feature importance	Horizontal bar chart	Random Forest impurity importance aggregated per feature.
Model comparison	Predicted vs actual scatter, both models	Both models against the ideal 1:1 line on held-out data.

25. Flask - the web framework

Flask is a micro-framework: it supplies routing, request/response handling and templating, and nothing else. That minimalism is the reason it was chosen - every line of request handling in this project is visible and readable, with no framework magic to reason around.

Routing

Routes are declared with decorators. The application separates page routes, which return HTML, from `/api/*` routes, which return JSON and are intended for programmatic access. The API routes accept both JSON bodies and form-encoded bodies, so they can be exercised with `curl`, a script, or a plain HTML form.

Request and response objects

`request.args` exposes query parameters (used by `/result` and by pre-filling the form), `request.get_json(silent=True)` parses a JSON body without raising on malformed input, and `jsonify()` serialises responses with the correct content type. Method restrictions are enforced by Flask: `GET /api/predict` returns 405 by design.

Jinja2 templating

`base.html` defines the shell - navigation, footer, stylesheet and script tags - and each page fills a `content` block. This removes all duplicated markup. Jinja2 escapes substituted values by default, so user-supplied strings cannot inject markup. Four custom template filters keep formatting consistent across every page: `currency`, `currency2`, `num` and `signed_currency`.

Error handling

Handlers are registered for 404, 405 and 500, plus explicit checks for the cases that matter here - missing model artifact, no parameters supplied, parameters that fail validation. Each renders a specific page with an explanation and a way out. Server-side logging records full detail including stack traces, while the browser receives only a plain message; no internal path, variable or traceback is ever exposed.

Built-in development server

Running `app.py` starts Flask's own server bound to `0.0.0.0:5000`. This is a development tool, not a production server - it is single-process and not hardened. The README states plainly that a WSGI server such as Gunicorn or Waitress should be used for anything beyond local demonstration.

26. Frontend technologies

HTML5

Markup is semantic throughout: header, nav, main, aside and footer elements rather than generic containers. Every input has an associated label, radio groups are wrapped in a `radiogroup` role, and tables use caption, thead and scope attributes so assistive technology can navigate them.

CSS3

No CSS framework is loaded. A single stylesheet of roughly 1,100 lines defines the entire visual system, starting with a block of custom properties (design tokens) for colour, spacing, radii and shadows, so a change to the palette propagates everywhere. Layout uses CSS Grid for page-level structure and Flexbox for components. Three breakpoints adapt the layout for tablet, phone and small phone; the `prefers-reduced-motion` media

query disables transitions for users who request it; and a print stylesheet removes navigation so pages can be printed for a project submission.

The visual direction is deliberately restrained: a warm off-white page, charcoal text, a forest-green accent, hairline borders, 8-12 px radii and shadows used only where a surface genuinely needs separating from the page. There are no gradients, no glassmorphism, no emoji icons and no decorative cards.

Vanilla JavaScript

One 300-line file, no dependencies and no build step. It handles three things: the mobile navigation toggle, client-side validation that mirrors the server rules, and the Fetch API submission with loading and error states.

The submission path is a good illustration of progressive enhancement. The form in HTML is a working GET form pointing at `/result`. If JavaScript is unavailable or the Fetch call fails outright, the browser falls back to a normal submission and the server produces the same result. JavaScript, when present, intercepts the submit, validates, calls the API, and then navigates to the identical URL - so the fast path and the fallback path converge on the same page.

Accessibility is handled in the small details: a skip link to main content, visible focus-visible outlines on every interactive element, an `aria-live` status region announcing validation and prediction state, and `aria-expanded` kept in sync on the navigation toggle.

27. pytest - the testing stack

The suite contains 49 tests across two files, all passing. `conftest.py` inserts the project root onto `sys.path` so imports work regardless of the directory pytest is invoked from, exposes a shared `sample_input` fixture, and defines a `requires_artifacts` marker that skips training-dependent tests with a clear message if the model has not been built yet - a skip is distinguished from a failure, which keeps a fresh clone informative rather than alarming.

Flask is tested through its own `app.test_client()`, which exercises the real routing, validation and error-handling code without needing a running server. This means the tests cover the actual request path, not a reimplementation of it.

Part V - Results

28. Model comparison

Both candidates were fitted on the same 1,166 training rows and scored on the same untouched 292 test rows. Every number below is read from `models/metrics.json`.

Metric	Split	Random Forest	Linear Regression	Winner
MAE	Test	$19,039	$21,974	Forest by $2,935
RMSE	Test	$27,041	$30,379	Forest by $3,338
R2	Test	0.8676	0.8329	Forest by +0.0347
MAPE	Test	12.18%	13.87%	Forest by 1.69 pts
MAE	Train	$8,325	$21,104	-
R2	Train	0.9743	0.8561	-

The Random Forest wins on every held-out metric, so it is the deployed model. The selection is performed in code by comparing test RMSE - not hard-coded - and the reason is written into `metrics.json` and displayed on the Insights page:

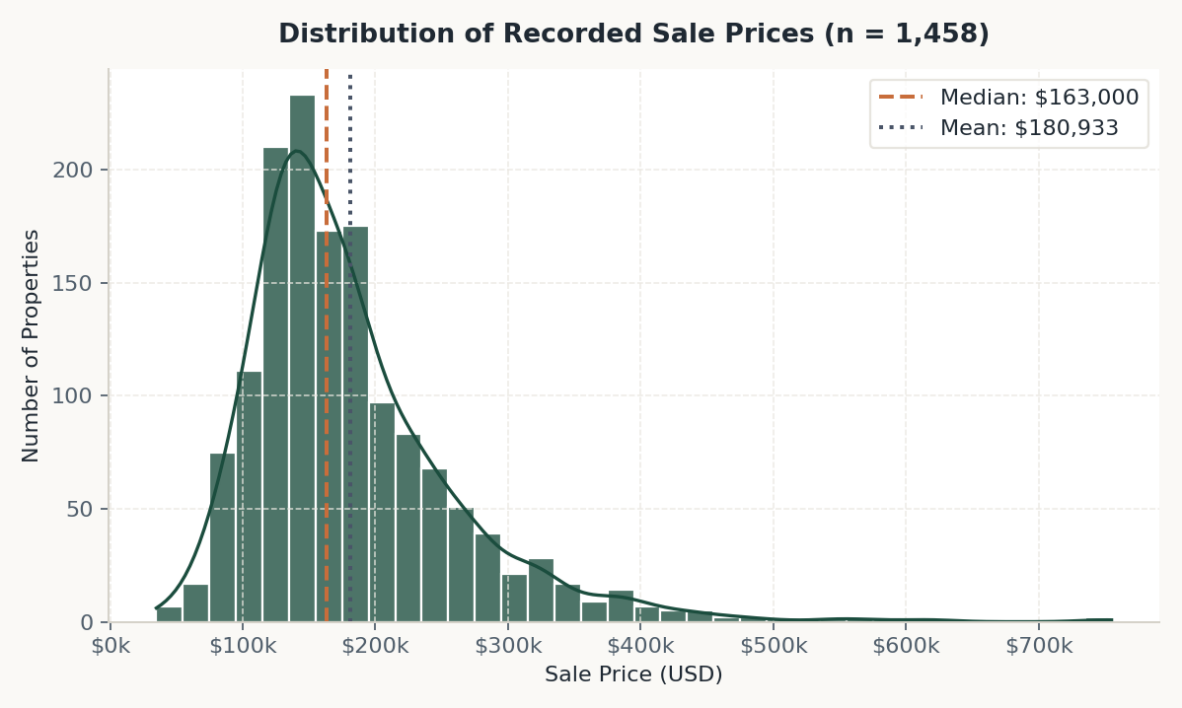
> **Selection reason recorded in metrics.json**
> Random Forest Regressor achieved lower held-out test error (RMSE $27,040.66 vs. $30,378.88, MAE $19,039.07 vs. $21,974.48) and higher explained variance (R^2 0.8676 vs. 0.8329).

The practical reading: moving from the linear baseline to the forest cuts typical error from $21,974 to $19,039 - a 13.4% improvement - and lifts explained variance from 83.3% to 86.8%. That gain comes from the forest's ability to express non-linearities and feature interactions that a linear equation cannot represent.

29. Charts

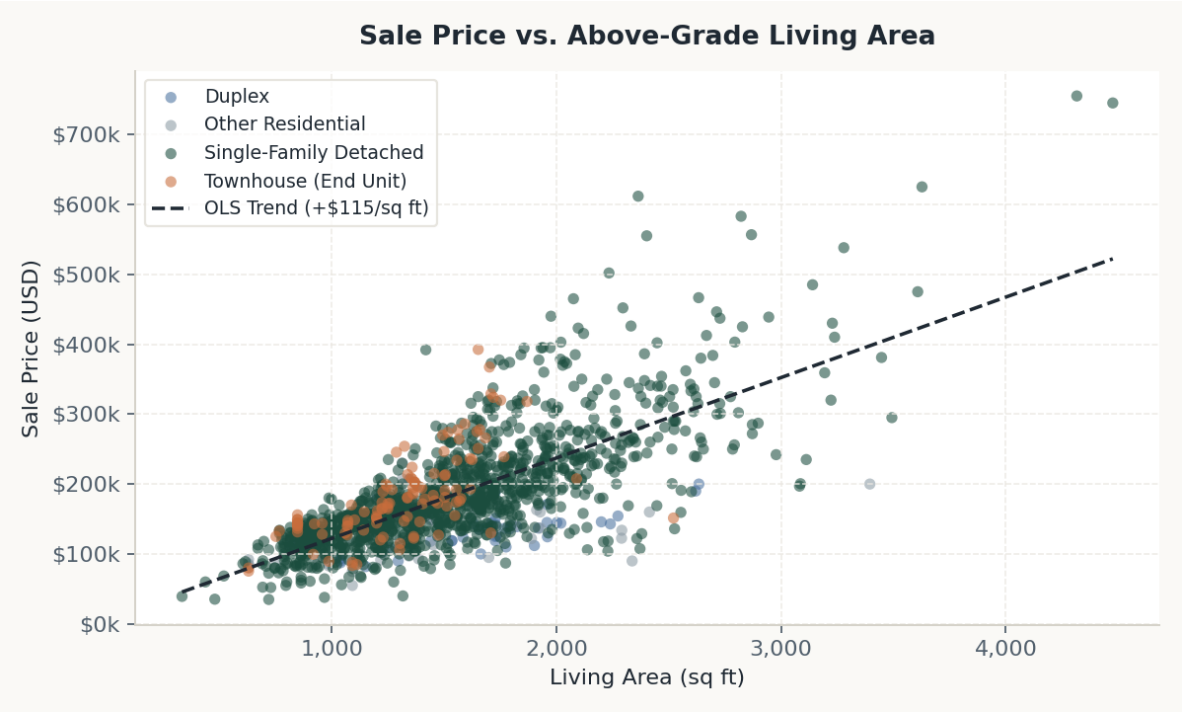
The six figures below are the actual PNG files served by the Insights page.

29.1 Sale price distribution



Histogram with kernel density estimate. Median and mean are marked; the right tail drives the gap between MAE and RMSE.

29.2 Price versus living area



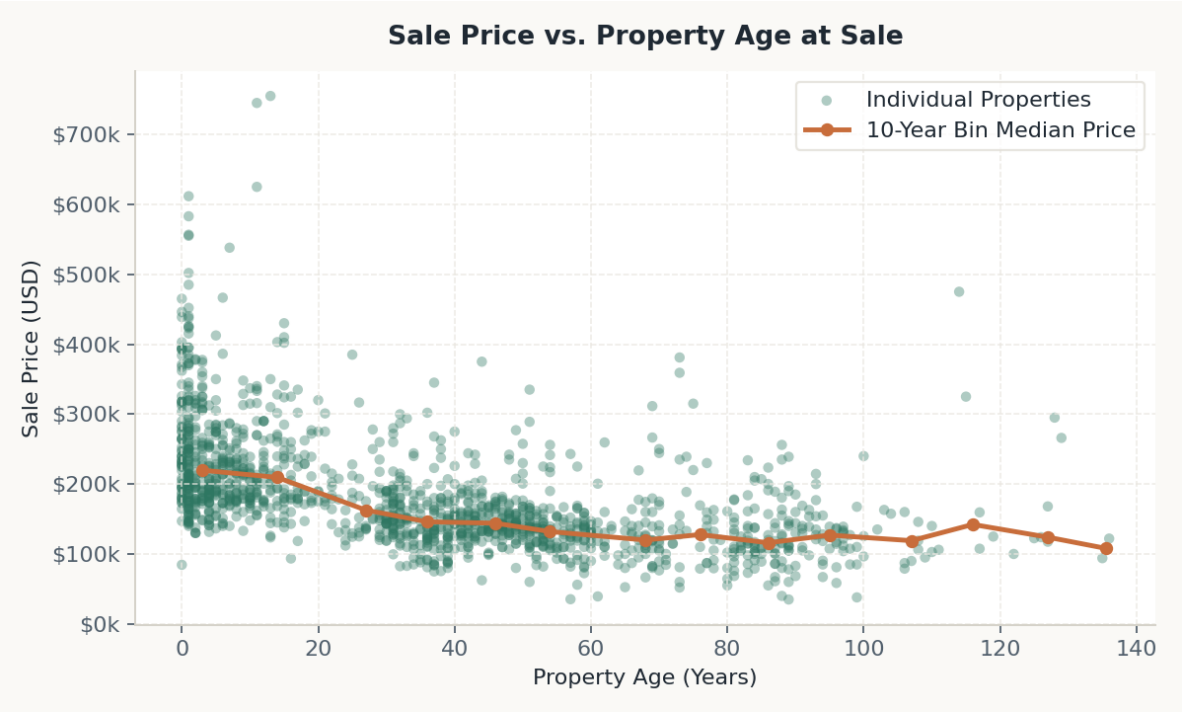
Scatter by property type with an ordinary least squares trend line and its slope in dollars per square foot.

29.3 Price versus bedrooms



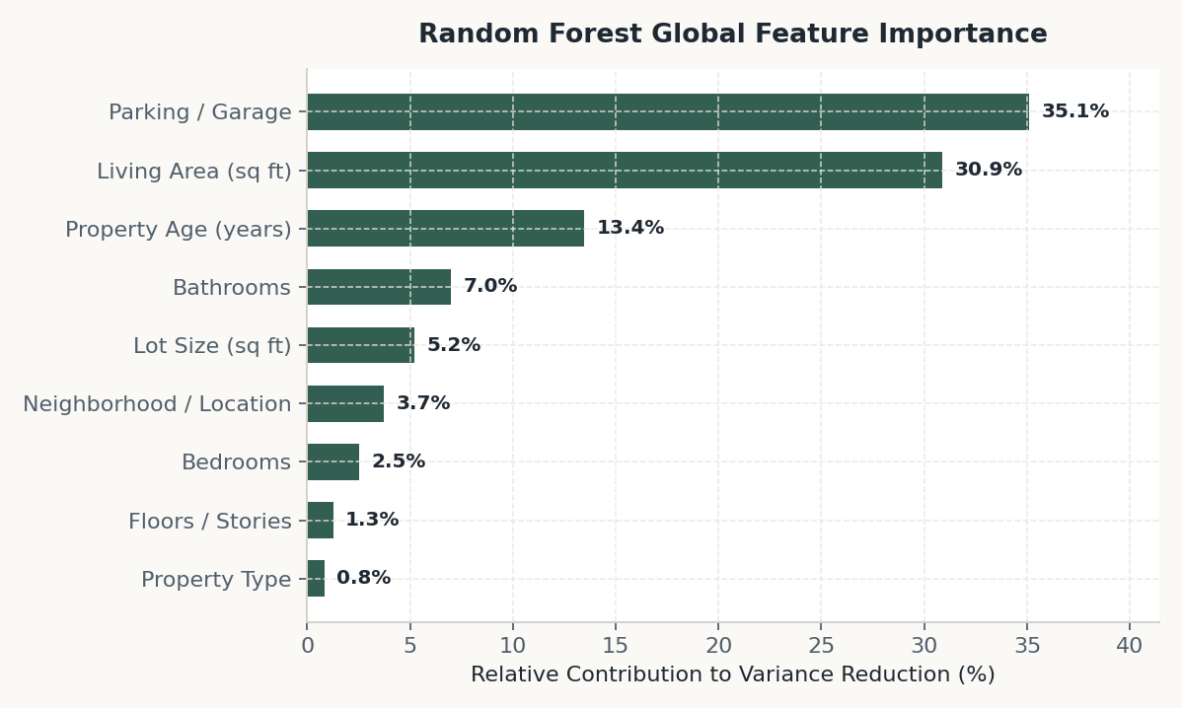
Box plot by above-grade bedroom count. The widening spread at higher counts explains why bedrooms carry low feature importance.

29.4 Price versus property age



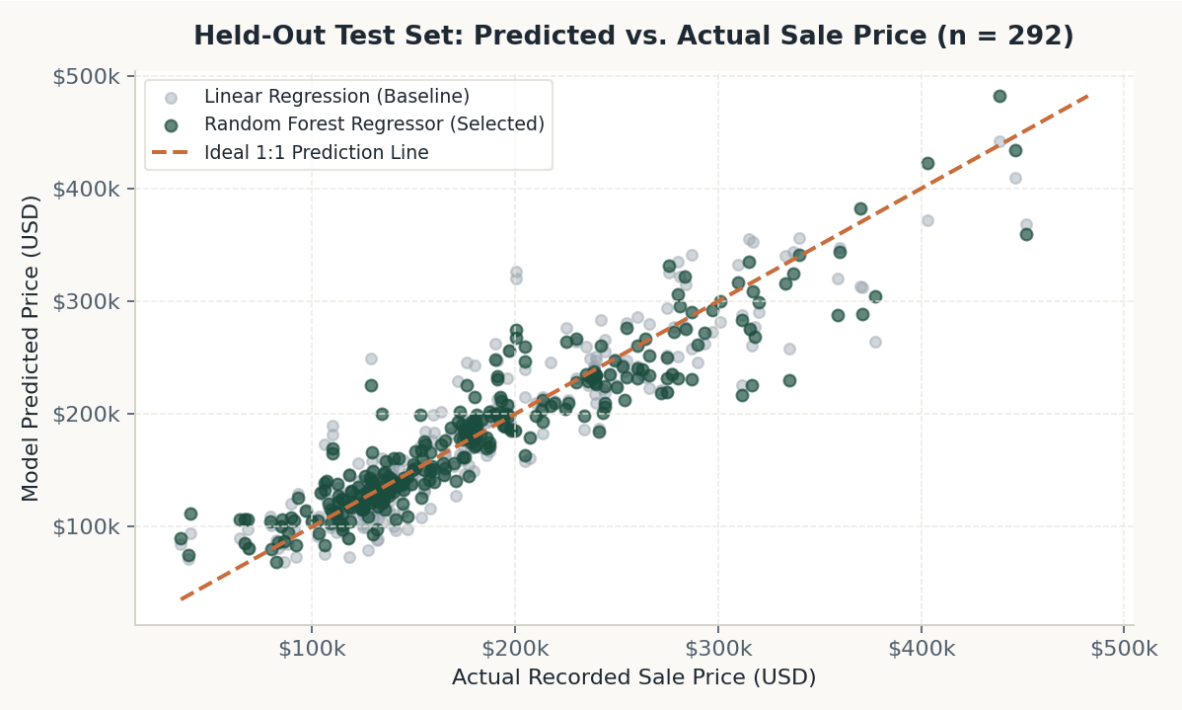
Individual sales with the median price per ten-year age band overlaid. The relationship is non-linear, not a straight decline.

29.5 Global feature importance



Random Forest impurity importance, with one-hot blocks summed back onto their source features.

29.6 Predicted versus actual on held-out data



Both models against the ideal 1:1 line. Points below the line are under-estimates; the forest clusters more tightly.

Part VI - Application Reference

30. Routes and the API contract

Route	Methods	Returns	Notes
/	GET	HTML	Landing page with live dataset facts.
/predict	GET	HTML	Form; accepts query parameters to pre-fill fields.
/result	GET	HTML	Recomputes the estimate from query parameters. 400 on invalid input, 503 if the model is missing.
/insights	GET	HTML	Charts, statistics and the recorded training run.
/api/predict	POST	JSON	Validates and predicts. 400 invalid, 422 unusable, 503 model missing.
/api/metrics	GET	JSON	Saved metrics, importances, statistics, chart paths.
/health	GET	JSON	Model readiness probe.

Successful prediction response (abridged)

```
{
  "success": true,
  "predicted_price": 239327.70,
  "currency": "USD",
  "currency_symbol": "$",
  "model_name": "Random Forest Regressor",
  "model_metrics": { "mae": 19039.07, "rmse": 27040.66,
                     "r2": 0.8676, "mape_percent": 12.18 },
  "input_summary": { "location": "Northridge Heights", ... },
  "explanation": {
    "predicted_price_per_sqft": 113.97,
    "market_median_price": 163000.0,
    "difference_vs_market_median": 76328.0,
    "price_band": "within the upper quartile of recorded sale prices",
    "versus_training_medians": [ ... ],
    "sensitivity": [ ... ],
    "top_global_features": [ ... ]
  },
  "warnings": [],
  "unseen_categories": [],
  "disclaimer": "This estimate is generated by a machine-learning model ..."
}
```

Error response

```
{
  "success": false,
  "error": "Location 'Atlantis' is not one of the neighborhoods present
            in the training data. Parking availability is required. ..."
}
```

The response shape is identical for every failure: `success: false` plus a human-readable `error` string. All validation problems are collected and returned together, so a user with three bad fields fixes them in one round trip rather than three.

31. Validation rules

Rules are declared once in `config.VALIDATION_RULES` and consumed in three places: server-side validation, the rendered HTML attributes, and the JavaScript object injected into the page. There is one source of truth.

Field	Minimum	Maximum	Default
Living area (sq ft)	300	10,000	1,464 (training median)
Lot area (sq ft)	1,000	250,000	9,550 (training median)
Bedrooms	1	10	3 (training median)
Bathrooms	1	8	2 (training median)
Property age (years)	0	160	36 (training median)
Floors	1	3	1 (training median)

Two cross-field rules also apply:

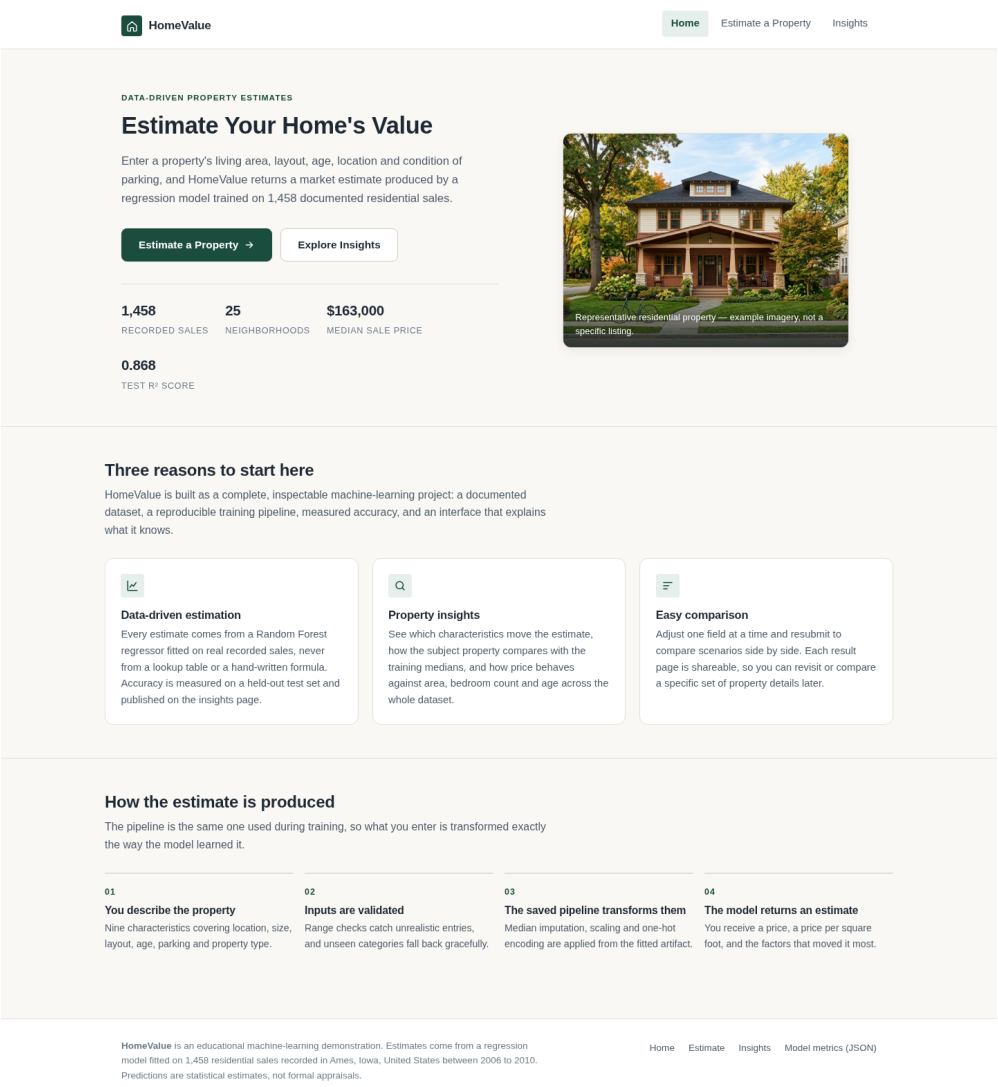
- Living area must be at least 100 sq ft per bedroom - a 400 sq ft four-bedroom house is almost certainly a typo, and the model was never fitted on anything like it.
- Lot area must not be smaller than living area - physically impossible, and it would push the model outside its training range.

Bounds are deliberately set close to the observed data range. A prediction for a 50,000 sq ft mansion would be pure extrapolation - a Random Forest cannot extrapolate beyond values it saw, so it would silently return the price of the largest house it knows. Rejecting the input is more honest than returning a number that looks confident but is meaningless.

32. User interface

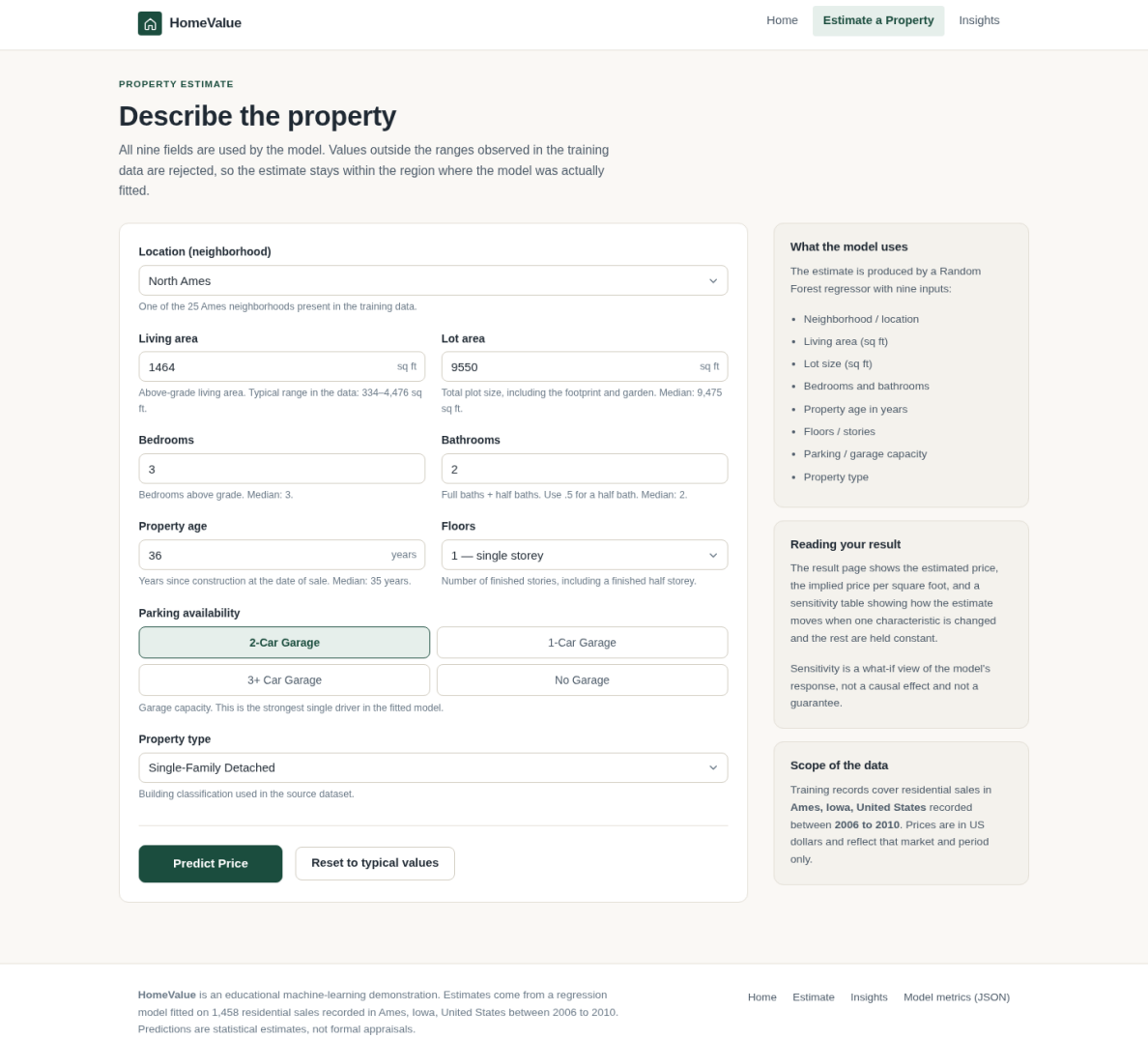
The interface is built around one principle: every element earns its place. The screens below are the actual rendered application.

32.1 Landing page



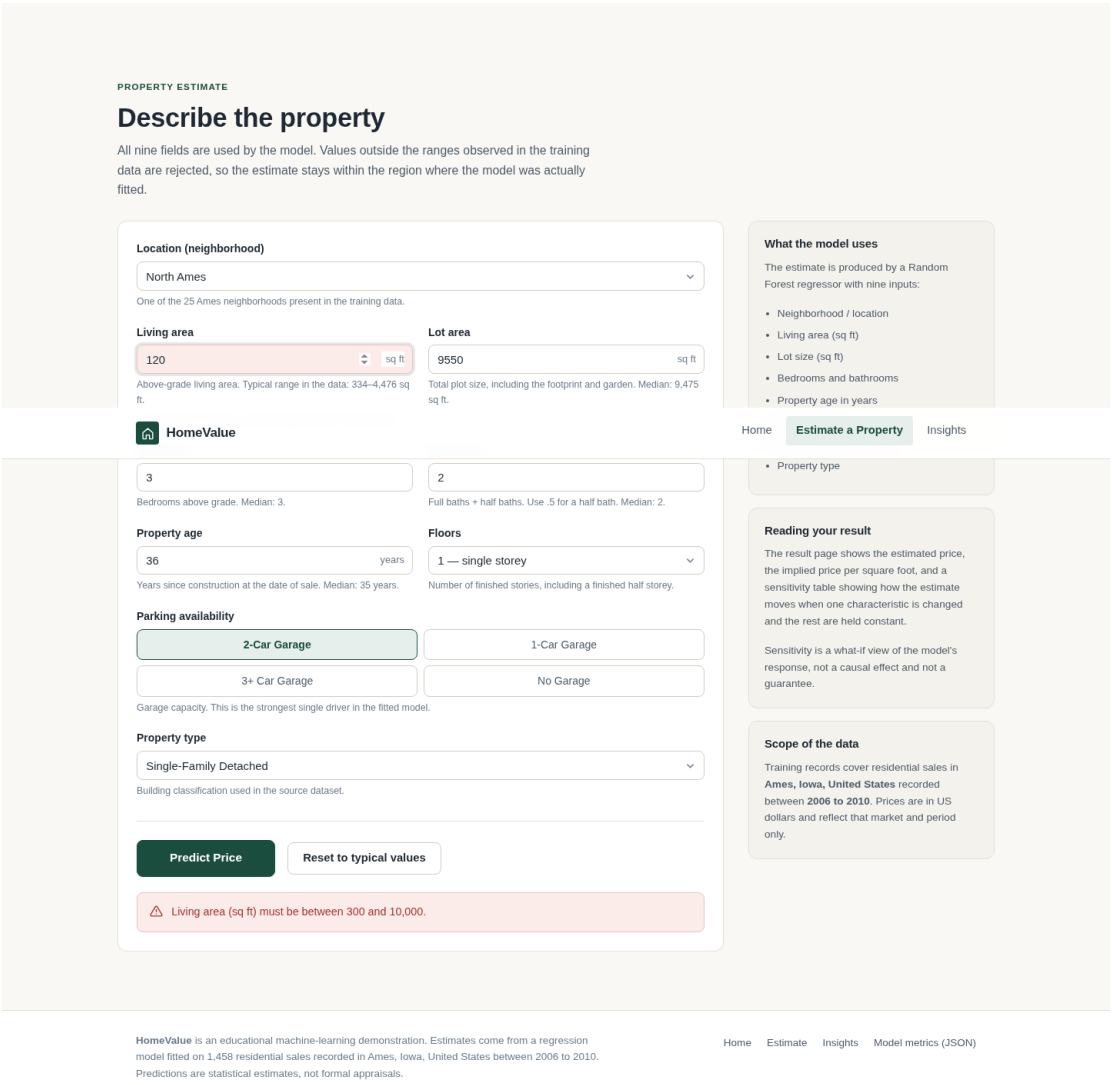
Headline, two calls to action, live dataset facts and an editorial property visual.

32.2 Estimate form



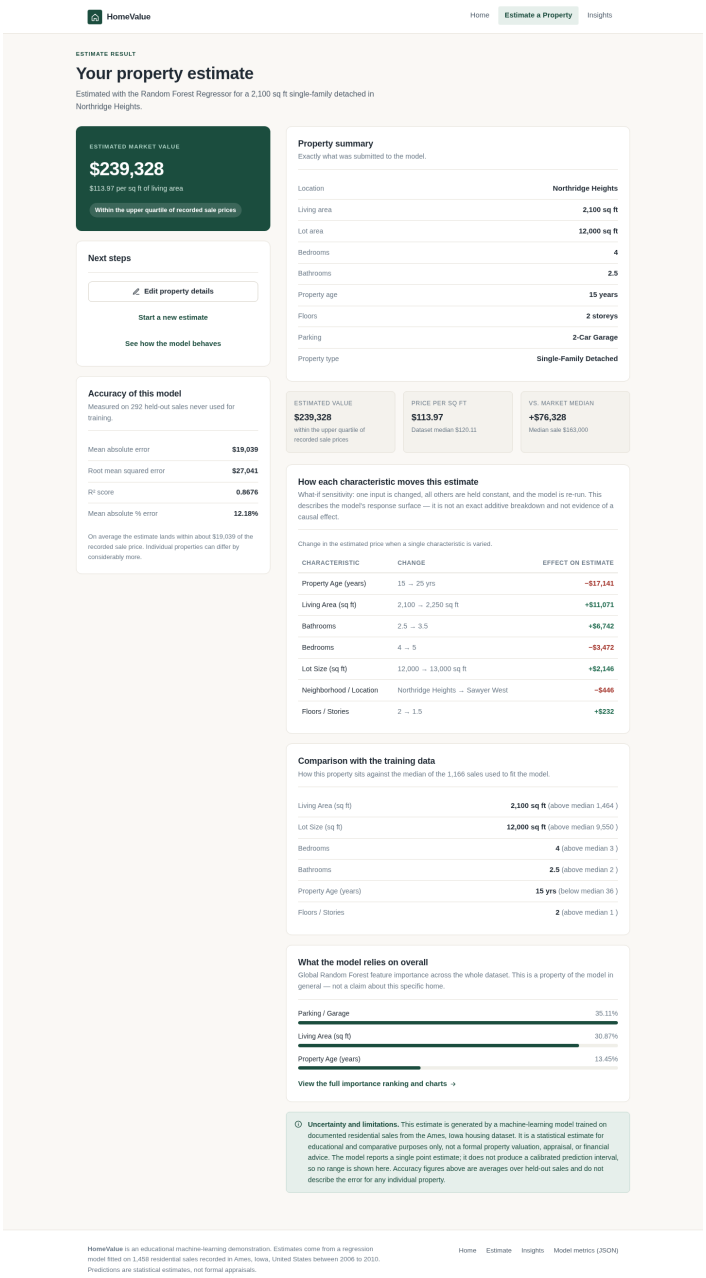
Nine inputs with dataset-grounded hints; defaults are the training medians so the form is usable immediately.

32.3 Validation feedback



Field-level errors plus a summary banner. Identical rules run on the server.

32.4 Result page



Estimated price, price per square foot, sensitivity table, comparison with training medians and the limitations notice.

32.5 Insights page



Feature importance in detail

Two independent measures are reported, because they answer different questions. Impurity importance reflects how often a feature was used to split trees, which tends to favour high-cardinality features. Permutation importance measures the drop in R^2 on unseen data when a feature's values are shuffled, which is closer to predictive usefulness.

FEATURE	IMPURITY IMPORTANCE	PERMUTATION IMPORTANCE (SHARE OF TOTAL)
Parking / Garage	35.11%	9.36%
Living Area (sq ft)	30.87%	48.7%
Property Age (years)	13.45%	26.65%
Bathrooms	6.99%	3.66%
Lot Size (sq ft)	5.23%	3.3%
Neighborhood / Location	3.73%	2.08%
Bedrooms	2.54%	3.29%
Floors / Stories	1.26%	2.22%
Property Type	0.83%	0.79%

Interpretation caution. These are global association measures computed from the fitted model and the historical data. They describe which inputs the model relies on, not which factors cause a price. Feature importance should not be read as advice about which improvements would raise a home's value.

Dataset statistics

Source: Ames Iowa Residential Housing Dataset, compiled by Dean De Cock (Truman State University) and published in Journal of Statistics Education, Vol. 19, No. 3 (2011). Retrieved from OpenML (dataset 42165).

Sale price

Minimum	$34,900
25th percentile	$129,925
Median	$163,000
Mean	$180,933
75th percentile	$214,000
Maximum	$755,000
Standard deviation	$79,495

Property characteristics

Living area (median)	1,462 sq ft
Living area (range)	334–4,476 sq ft
Lot area (median)	9,478 sq ft
Lot area (range)	1,300–215,245 sq ft
Bedrooms (median)	3
Bathrooms (median)	2
Age at sale (median)	35 yrs
Age at sale (range)	0–136 yrs

Median sale price by neighborhood

All 25 neighborhoods present in the training data, ranked by median sale price.

NEIGHBORHOOD	SALES	MEDIAN PRICE	MEAN PRICE	MEDIAN AREA (SQ FT)	MEDIAN AGE (YRS)
Northridge Heights	77	$315,000	$316,271	1,890	1.0
Northridge	41	$301,500	$305,295	2,418	12.0
Stone Brook	25	$278,000	$310,499	1,742	7.0
Timberland	38	$228,475	$242,247	1,680	8.0
Somerset	86	$225,500	$225,360	1,564	1.0
Veenker	11	$218,000	$238,773	1,437	29.0
Crawford	51	$200,624	$210,625	1,717	69.0
Clear Creek	28	$200,250	$212,565	1,738	41.5
College Creek	150	$197,200	$197,966	1,500	6.0
Bloomington Heights	17	$191,000	$194,671	1,500	2.0
Northwest Ames	73	$182,900	$189,090	1,664	32.0
Gilbert	79	$181,000	$192,855	1,593	6.0
Sawyer West	59	$179,900	$186,556	1,603	17.0
Mitchell	49	$159,500	$156,270	1,204	26.0
Northpark Villa	9	$146,000	$142,694	1,322	32.0
North Ames	225	$140,000	$145,847	1,200	48.0
South & West of ISU	25	$139,500	$142,591	1,691	83.0
Bluestem	2	$137,500	$137,500	1,392	28.5
Sawyer	74	$135,000	$136,793	1,106	43.0
Brookside	56	$124,300	$124,834	1,210	78.0
Edwards	98	$119,500	$127,319	1,198	54.0
Old Town	113	$119,000	$128,225	1,374	86.0
Briardale	16	$106,000	$104,494	1,195	37.0
Iowa DOT & Rail Road	37	$103,000	$100,124	1,128	84.0
Meadow Village	17	$88,000	$96,576	1,092	36.0

Median sale price by property type

PROPERTY TYPE	SALES	MEDIAN PRICE	MEDIAN AREA (SQ FT)
Townhouse (End Unit)	114	$172,200	1,337
Single-Family Detached	1218	$167,900	1,482
Townhouse (Interior)	43	$137,500	1,306
Duplex	52	$139,950	1,560
Two-Family Conversion	31	$127,500	1,367

Data preparation record

Raw records loaded	1,460 rows × 19 columns
Duplicate rows removed	0
Invalid or anomalous sales removed	2
Records remaining after cleaning	1,458
Missing values handled by the pipeline	Median (numeric) / most frequent (categorical), fitted on the training split only
Training rows	1,166
Test rows (held out)	292
Model last trained	2026-10-26T17:06:56.583907+00:00

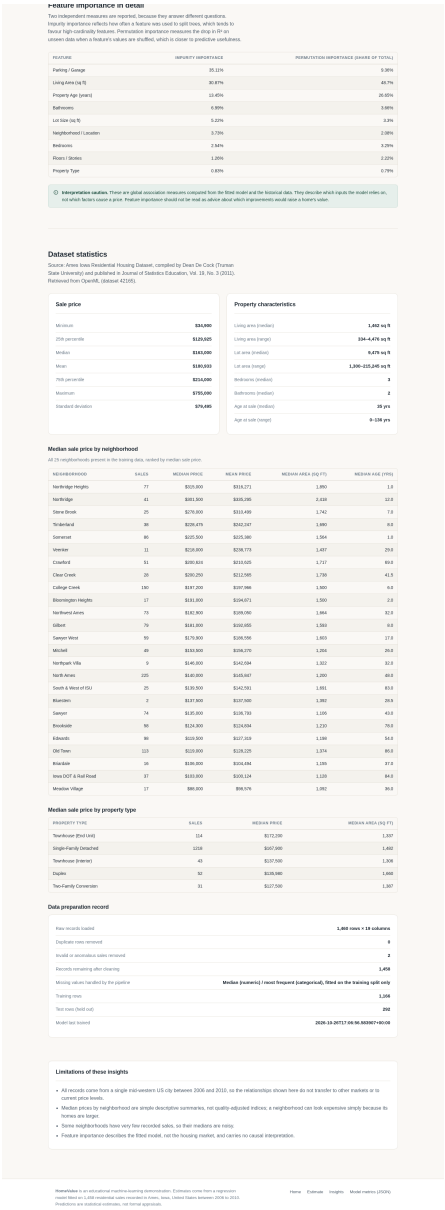
Limitations of these insights

- All records come from a single mid-western US city between 2006 and 2010, so the relationships shown here do not transfer to other markets or to current price levels.
- Median prices by neighborhood are simple descriptive summaries, not quality-adjusted indices; a neighborhood can look expensive simply because its homes are larger.
- Some neighborhoods have very few recorded sales, so their medians are noisy.
- Feature importance describes the fitted model, not the housing market, and carries no causal interpretation.

HomeValue is an educational machine-learning demonstration. Estimates come from a regression model fitted on 1,458 residential sales recorded in Ames, Iowa, United States between 2006 to 2010. Predictions are statistical estimates, not formal appraisals.

Home Estimate Insights Model metrics (JSON)

Insights page, upper part: the statistics strip and the four distribution charts.



Feature importance in detail

Two independent measures are reported, because they answer different questions. Impurity importance reflects how often a feature was used to split trees, which tends to favour high-cardinality features. Permutation importance measures the drop in R² on unseen data when a feature's values are shuffled, which is closer to predictive usefulness.

FEATURE	IMPURITY IMPORTANCE	PERMUTATION IMPORTANCE (SHARE OF TOTAL)
Parking / Garage	35.11%	9.96%
Living Area (sq ft)	30.87%	48.7%
Property Age (years)	13.45%	26.65%
Bathrooms	6.99%	3.98%
Lot Size (sq ft)	5.23%	3.3%
Neighborhood / Location	3.75%	2.08%
Bedrooms	2.94%	3.29%
Floors / Stories	1.26%	2.22%
Property Type	0.83%	0.79%

Interpretation caution. These are global association measures computed from the fitted model and the historical data. They describe which inputs the model relies on, not which factors cause a price. Feature importance should not be read as advice about which improvements would raise a home's value.

Dataset statistics

Source: Ames Iowa Residential Housing Dataset, compiled by Dean De Cock (Truman State University) and published in Journal of Statistics Education, Vol. 19, No. 3 (2011). Retrieved from OpenML (dataset 42165).

Sale price

Minimum	$34,900
25th percentile	$129,925
Median	$163,000
Mean	$180,933
75th percentile	$214,000
Maximum	$755,000
Standard deviation	$79,495

Property characteristics

Living area (median)	1,462 sq ft
Living area (range)	334–4,476 sq ft
Lot area (median)	9,478 sq ft
Lot area (range)	1,300–215,245 sq ft
Bedrooms (median)	3
Bathrooms (median)	2
Age at sale (median)	35 yrs
Age at sale (range)	0–136 yrs

Median sale price by neighborhood

All 25 neighborhoods present in the training data, ranked by median sale price.

NEIGHBORHOOD	SALES	MEDIAN PRICE	MEAN PRICE	MEDIAN AREA (SQ FT)	MEDIAN AGE (YRS)
Northridge Heights	77	$325,000	$316,271	1,890	1.0
Northridge	41	$301,500	$305,295	2,418	12.0
Stone Brook	25	$278,000	$310,459	1,742	7.0
Timberland	36	$239,479	$242,197	1,660	9.0
Somerset	85	$225,500	$225,379	1,564	1.0
Veenker	11	$218,000	$238,773	1,437	29.0
Crawford	51	$200,624	$210,625	1,717	69.0
Clear Creek	26	$200,250	$212,565	1,755	41.5
College Creek	180	$197,200	$197,966	1,900	6.0
Bloomington Heights	17	$191,000	$194,671	1,590	2.0
Northwest Ames	73	$182,900	$186,090	1,864	32.0
Gilbert	79	$181,000	$192,655	1,569	8.0
Sawyer West	59	$179,900	$186,556	1,603	17.0
Mitchell	49	$159,500	$156,270	1,256	39.0
Northpark Villa	9	$146,900	$140,694	1,322	32.0
North Ames	225	$140,000	$145,667	1,260	48.0
South & West of ISU	25	$139,500	$142,591	1,091	83.0
Blueste	2	$137,500	$137,500	1,092	28.5
Sawyer	74	$135,000	$136,793	1,106	43.0
Brookside	56	$124,300	$124,934	1,230	70.0
Edwards	99	$119,500	$127,319	1,194	54.0
Old Town	113	$119,000	$126,225	1,374	86.0
Iowa DOT & Rail Road	37	$103,600	$109,124	1,120	84.0
Meadow Village	17	$88,000	$98,576	1,092	36.0

Median sale price by property type

PROPERTY TYPE	SALES	MEDIAN PRICE	MEDIAN AREA (SQ FT)
Townhouse (End Unit)	114	$172,500	1,337
Single-Family Detached	1218	$167,000	1,462
Townhouse (Interior)	43	$137,500	1,308
Duplex	52	$129,500	1,468
Two-Family Conversion	31	$127,500	1,367

Data preparation record

Raw records loaded	1,460 rows × 19 columns
Duplicate rows removed	0
Invalid or anomalous sales removed	2
Records remaining after cleaning	1,458
Missing values handled by the pipeline	Median (numeric) / most frequent (categorical), fitted on the training split only
Training rows	1,166
Test rows (held out)	292
Model last trained	2026-10-26T17:06:56.863907+00:00

Limitations of these insights

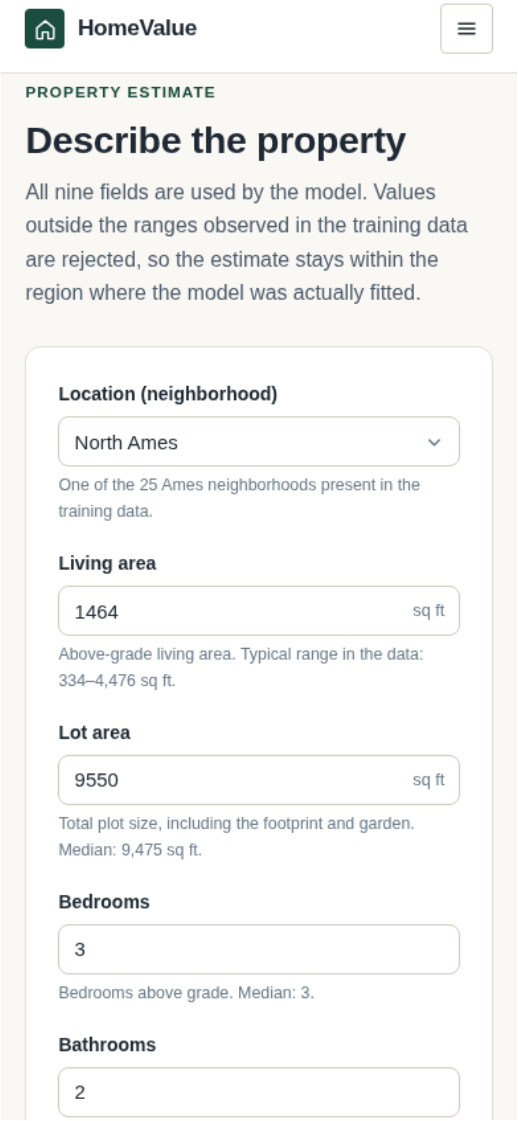
- All records come from a single mid-western US city between 2006 and 2010, so the relationships shown here do not transfer to other markets or to current price levels.
- Median prices by neighborhood are simple descriptive summaries, not quality-adjusted indices; a neighborhood can look expensive simply because its homes are larger.
- Some neighborhoods have very few recorded sales, so their medians are noisy.
- Feature importance describes the fitted model, not the housing market, and carries no causal interpretation.

HomeValue is an educational machine-learning demonstration. Estimates come from a regression model fitted on 1,458 residential sales recorded in Ames, Iowa, United States between 2006 to 2010. Predictions are statistical estimates, not formal appraisals.

Home · Estimate · Insights · Model metrics (JSON)

Insights page, lower part: model comparison, feature importance table and dataset statistics.

32.6 Mobile layout



The same form at a 390 px viewport, with a collapsible navigation menu.

Part VII - Engineering Quality

33. Test coverage

49 tests, all passing. They are grouped by what they protect.

test_preprocessing.py

- The raw dataset loads with the expected shape, columns and known missing values.
- Cleaning maps to the exact nine-feature schema and produces 1,458 rows.
- The two documented anomalous sales are removed and no invalid values survive.
- Every code is mapped to a readable label within the declared option sets.
- The fitted preprocessor removes all NaNs - imputation demonstrably works.
- Unseen categories (a fictional "Atlantis", "Helicopter Pad", "Houseboat") transform without raising and without producing NaN.
- Numeric output is standardised to zero mean and unit variance; one-hot output contains only 0 and 1.
- Cleaning accepts an already-standardised frame and rejects an incomplete one.

test_prediction.py

- The saved Joblib pipeline loads and carries a metadata bundle whose feature schema matches the one in `config`.
- Metrics in `metrics.json` are internally consistent (RMSE at least MAE, R2 in range) and the recorded selected model really is the one with the lowest test RMSE.
- Train and test row counts sum to the cleaned total.
- Valid input converts to the expected schema; missing, non-numeric, out-of-range and unknown-location inputs each raise a specific error.
- Unseen parking and property types pass with warnings rather than failing.
- Comma-separated and string-encoded numbers are accepted - real users type "2,100".
- Predictions are finite, positive, floats and deterministic across repeated calls.
- Larger area predicts higher; newer predicts higher than much older - sanity checks on the model's direction, not just its arithmetic.
- Price per square foot equals the estimate divided by the submitted area.
- Batch prediction matches single prediction exactly.
- Every page route returns 200; an unknown route returns 404.
- The API accepts JSON and form bodies, rejects empty and non-JSON bodies, rejects GET, and returns the documented response shape.
- `/result` renders for a valid query, returns 400 for an invalid one and 400 when no parameters are supplied.
- No stack trace, file path or internal detail appears in any error response.

34. Reliability and edge cases

Situation	Behaviour
Model artifact missing	Buttons disabled, a warning shown on the landing page, 503 from the API, 503 on /result. The user is told the exact command to run.
Unseen categorical value	Encoded as all-zeros by the encoder; the result page shows a warning and the API lists the affected fields in unseen_categories.

Situation	Behaviour
Missing numerical value	Filled by the fitted SimpleImputer with the training median - the same treatment used during training.
Out-of-range value	Rejected with a message naming the field and the permitted range.
JavaScript disabled	The form submits as a normal GET; the server validates and renders the same result page.
Network failure during the Fetch call	A message is shown and the plain form submission is retried automatically.
Malformed JSON body	get_json(silent=True) returns None; the API responds 400 rather than raising.
Unexpected server error	Logged with a full trace server-side; the user sees a generic message. No internal detail is exposed.

35. Setup and operation

Four commands take a clean clone to a running application.

```
# 1. Create and activate a virtual environment
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1

# 2. Install dependencies
pip install -r requirements.txt

# 3. Train the model (writes models/, metrics.json and the six charts)
python -m src.train_model

# 4. Start the application
python app.py                      # then open http://127.0.0.1:5000
```

Other entry points

Command	Purpose
python -m src.data_preprocessing	Inspect the raw data and write the cleaned CSV only.
python -m src.predict	Run one sample prediction from the command line and print the result.
python -m pytest tests/ -v	Run the 49-test suite.
jupyter notebook notebooks/	Open the executed exploratory analysis notebook.
python docs/generate_documentation.py	Rebuild this PDF from the live artifacts.

Sample prediction from the command line

```
$ python -m src.predict
Sample prediction (USD):
  Estimated price: $239,327.70
  Price per sq ft: $113.97
  Model: Random Forest Regressor  |  Test R2: 0.8676
  Top local sensitivities:
    Property Age (years): -17,141
    Living Area (sq ft): +11,071
    Bathrooms: +6,742
    Bedrooms: -3,472
```

36. Limitations

Stated plainly, because a system that hides its limits is not trustworthy.

Limitation	Consequence	Mitigation in the product
Single city, historical period	Ames, Iowa, 2006-2010. Prices are neither current nor transferable.	Stated on the landing page, the form sidebar, the result page and the insights page.
Not a valuation	The output is a statistical estimate, not an appraisal.	A disclaimer accompanies every prediction, in the UI and in the API response.
No prediction interval	The model emits a point estimate only.	No range is displayed. The result page explains why one is absent.
Average error is not individual error	MAE of $19,039 is an average; a specific property can be much further off.	The result page spells this out next to the metrics.
No extrapolation	A Random Forest cannot predict outside its training range - it returns the extreme value it knows.	Input bounds reject out-of-range values before they reach the model.
Thin categories	Some neighbourhoods have very few recorded sales.	Sales counts are published in the neighbourhood table so readers can judge.
Omitted features	Condition, renovation quality and interior finish are absent.	Documented; these sit in the columns the schema does not consume.
Overfitting headroom	Training R2 0.9743 versus test R2 0.8676.	Both figures are published; regularisation is the top item in section 37.
Development server	Flask's built-in server is not production-grade.	Documented in the README with Gunicorn/Waitress named as the alternative.

37. Future improvements

- **Regularisation.** Tune `max_depth`, `min_samples_leaf` and `max_features` via `RandomizedSearchCV`. Feature subsampling is currently off (`max_features=1.0`), so enabling it should decorrelate the trees and close the train/test gap identified in section 22.
- **Calibrated intervals.** Quantile regression forests or conformal prediction would give genuinely justified prediction ranges rather than the point estimate the application currently offers.
- **Log-target modelling.** Training on log(price) would tame the right skew and should improve both RMSE and MAPE, at the cost of needing a back-transformation bias correction.
- **Richer features.** The source dataset carries 81 columns. Overall quality, basement area and remodelling year are strong candidates; the schema and pipeline already support adding them without restructuring.
- **Gradient boosting.** HistGradientBoostingRegressor or XGBoost would be a natural third candidate to add to the comparison.
- **Engineering.** Docker packaging, a CI workflow running the test suite, saved scenario comparison, and a model-card / data-drift monitor.

38. Glossary

Term	Definition
Bagging	Bootstrap aggregating - training each model on a random resample of the data and averaging their predictions to reduce variance.
Baseline model	A simple reference model that any more complex approach must beat to justify its complexity.
Bootstrap sample	A sample of size n drawn with replacement; contains roughly 63.2% of the distinct original rows.
ColumnTransformer	A Scikit-learn object that applies different transformers to different subsets of columns.
Data leakage	Information from the test set reaching the model during training, inflating reported accuracy.
Estimator	Any Scikit-learn object implementing the fit/transform/predict contract.
Feature importance	A score describing how much a model relies on an input; not a causal effect.
Held-out test set	Data withheld from training and used only to measure generalisation.
Hyperparameter	A setting chosen before training (e.g. tree depth), not learned from data.
Imputation	Replacing missing values with a substitute such as the median or mode.
Joblib	A serialisation library optimised for large NumPy arrays; the recommended format for Scikit-learn models.
MAE / RMSE	Mean absolute error and root mean squared error - average error, and average error weighted toward large misses.
MAPE	Mean absolute percentage error - average error expressed as a percentage of the true value.
One-hot encoding	Representing a k-level category as k binary columns.
Out-of-bag	The ~36.8% of rows excluded from a tree's bootstrap sample.
Overfitting	Fitting training data so closely that performance on new data degrades.
Permutation importance	The drop in model performance when a feature's values are shuffled on held-out data.
Pipeline	A Scikit-learn object chaining preprocessing and a model into one estimator.
R2	Coefficient of determination - the share of variance explained, from 0 to 1.
Recursive binary splitting	The greedy top-down procedure a decision tree uses to grow.
Standardisation	Rescaling a feature to zero mean and unit variance.
Variance reduction	The regression split criterion: minimise the total squared deviation in the child nodes.
WSGI	The Python standard interface between web servers and applications, which Flask implements.

End of document. Every figure, table and chart above was generated from the project's saved artifacts at build time (2026-10-26 training run). Re-run `python docs/generate_documentation.py` to regenerate.